
Lecture 2 – Static Analysis

1 Hailstone Sequence

- سلسلة تبدأ من رقم n .
- لو n زوجي $n/2 \rightarrow$ ، لو فردي $n+1.3 \rightarrow$
- تستمر حتى الوصول إلى 1.
- مثال: $3 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$.
- هدف: فهم loops و conditions في البرمجة.

2 Computing Hailstones

- مثال Python و Java.
- Java تتطلب:
 - semicolons
 - parentheses
 - curly braces
- البرمجة هي شكل من أشكال التواصل مع البشر والكمبيوتر.

3 Types

- primitive types, stores the actual data directly in memory(int, long, double, boolean, char.)
- **Object Types:** String, Wrapper classes (Integer, Double), BigInteger, Arrays, Collections (List).
- object types, stores a memory address (reference)
- **Operations:** يمكن أن تكون:
 - operators (a + b)
 - methods (bigint.add(bigint2))

Lecture 2 – Static Analysis

functions (Math.sin(theta)). ○

4 Static Typing

- Java اللغة **statically typed**: أنواع المتغيرات معروفة عند compile time.
- Static typing يمنع أخطاء التعامل مع أنواع غير مناسبة قبل تشغيل البرنامج.
- Dynamically typed languages (مثل Python) تتحقق من الأنواع وقت التشغيل.

5 Static vs Dynamic Checking

- **Static checking**: الأخطاء يتم اكتشافها قبل تشغيل البرنامج (syntax, types, wrong names, wrong arguments).
- **Dynamic checking**: الأخطاء تظهر وقت التشغيل. (divide by zero, index out of range).

6 Static Analysis

- تحليل الكود بدون تنفيذه.
- الفوائد:
 - اكتشاف الأخطاء مبكرًا.
 - الأمان: كشف الثغرات.
 - التأكد من اتباع coding standards.
 - قياس تعقيد الكود.

7 Tools

- **Find Bugs**: يبحث عن مشاكل في bytecode ، يصنفها حسب الصعوبة من 1 إلى 20.
 - ↑ Scariest: ranked between 1 & 4. || ↑ Scary: ranked between 5 & 9. || ↑ Troubling: ranked between 10 & 14. || ↑ Of concern: ranked between 15 & 20.

Lecture 2 – Static Analysis

- **Checkstyle:** يركز على أسلوب البرمجة، indentation, naming, Javadoc, code complexity.

8 Primitive Types Issues

- Integer division: $5/2 = 2$ (مقطوعة).
- Integer overflow: عمليات تتجاوز الحد الأقصى/الأدنى للـ int أو long.
- Floating-point special values: NaN, POSITIVE_INFINITY, NEGATIVE_INFINITY.

9 Arrays and Lists

- **Arrays:** طول ثابت، لا يمكن تغييره بعد الإنشاء. | Arrays are fixed-length
- **Lists:** طول متغير، أفضل للاستخدام العملي. | Lists are variable-length
- في Java تستخدم object types ، مش primitives مباشرة.

10 Methods

- كل الكود في Java يجب أن يكون داخل method ، وكل method داخل class.
- **Comments** قبل method (توضح inputs, outputs, constraints مثل n must be positive).

11 Mutable vs Immutable Types

- **Immutable:** لا يمكن تغيير القيمة بعد الإنشاء. (String, int, Integer, Double)
- **Mutable:** يمكن تعديل القيمة. (ArrayList, StringBuilder, Date)

12 Snapshot Diagrams

- أداة لفهم العلاقة بين variables و references والقيم في الذاكرة.
- أي reassignment يغير السهم وليس القيمة القديمة.

Lecture 2 – Static Analysis

1[3] Mutating Values

- تغيير محتوى object mutable يسمى mutation.
- مثال list.set(0, 9) أو sb.append("b").

1[4] Immutable References (final)

- final يجعل المرجع ثابتًا.
- يمكن تغيير محتوى object mutable إذا كان المرجع final.

1[5] Aliasing

- نفس mutable object يمكن أن يكون له أكثر من reference.
- التعديل من أي reference يؤثر على جميع references.

1[6] Risks of Mutation

- مشاكل mutation تظهر عندما يكون هناك aliasing.
- يمكن أن تسبب bugs صعبة ومفاجئة.

1[7] Returning Mutable Objects

- إعادة mutable object مباشرة يمكن أن تؤدي إلى مشاكل.
- الحل: إعادة نسخة جديدة من object.

1[8] Iterating and Modifying Lists

- لا يمكن تعديل collection أثناء iteration باستخدام for-each.
- يؤدي إلى ConcurrentModificationException.

Lecture 2 – Static Analysis

1[9] Useful Immutable Types

- Primitive types و wrappers كلها immutable.
- BigInteger, BigDecimal, LocalDate immutable.
- Collections يمكن عمل unmodifiable wrappers لحماية البيانات.

2[0] Documenting Assumptions

- كتابة نوع المتغير توثق افتراض عن القيم.
- final يوضح أن المرجع لا يتغير.
- التعليقات توضح شروط إضافية) مثل (n must be positive).
- البرمجة هدفها: التواصل مع الكمبيوتر وفهم البشر للكود.